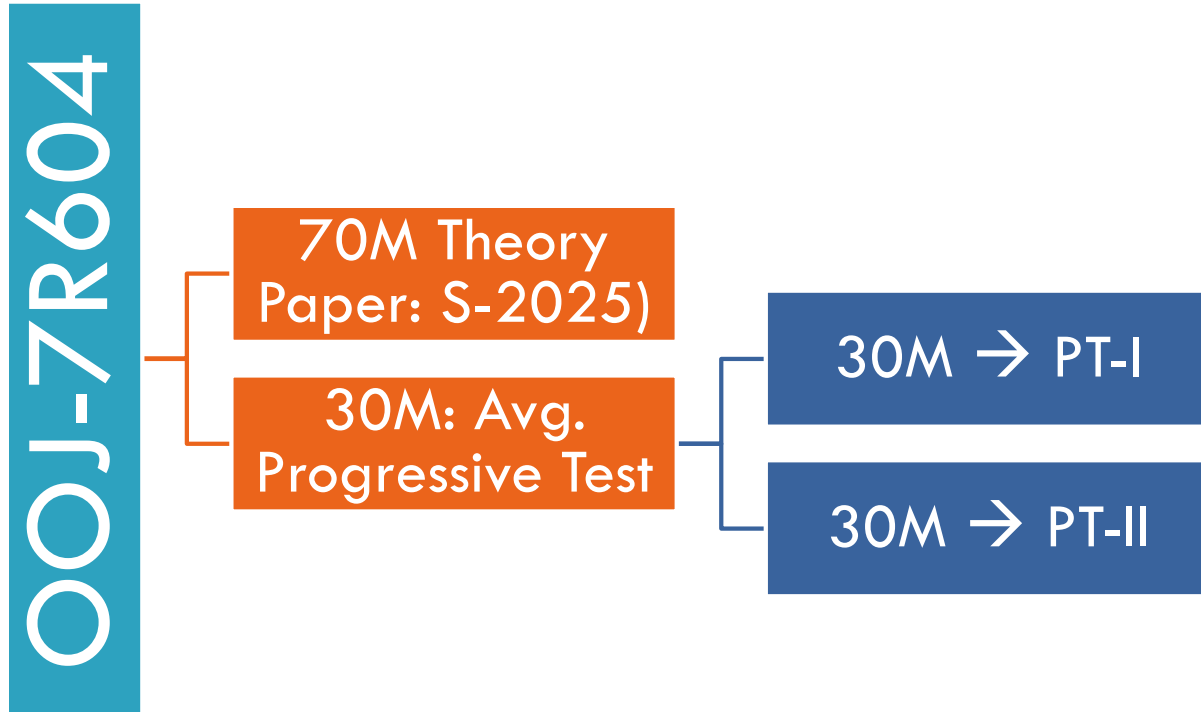




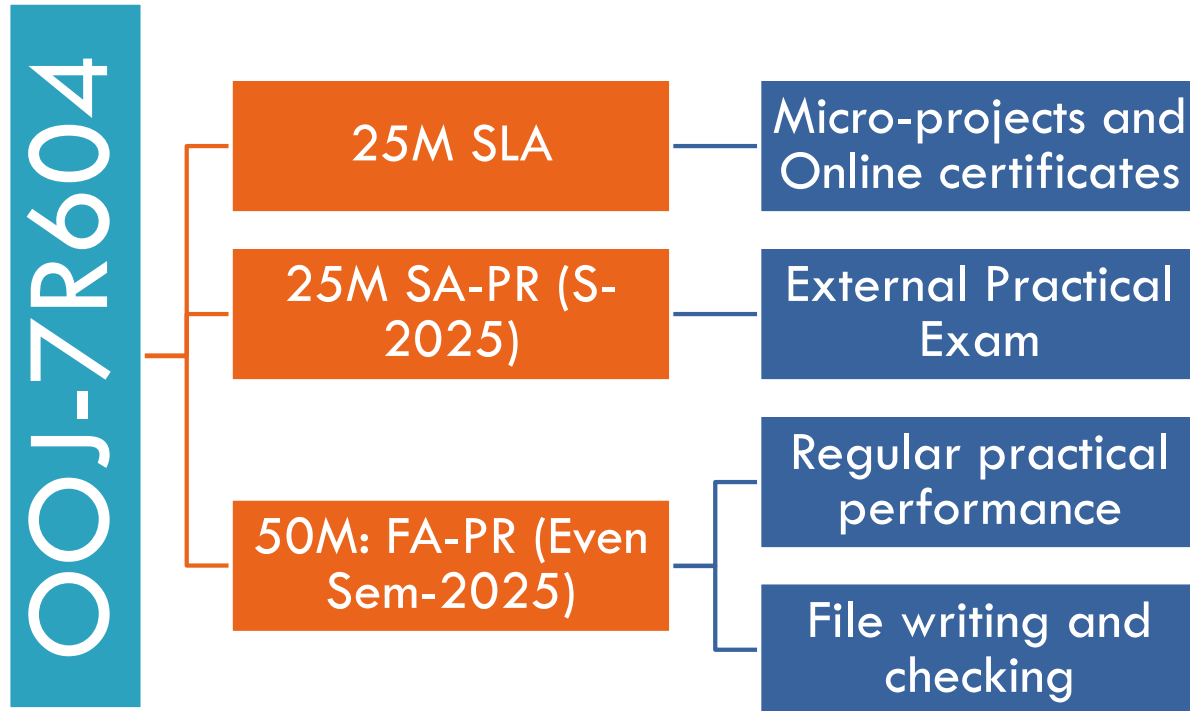
OBJECT ORIENTED PROGRAMMING USING JAVA (7R604-OOJ)

Dr. Nilesh Vishwasrao Patil

OOJ: 100 Marks (Theory)



OOJ: 100 Marks (Practical)



OOJ: 70 TH Marks

Chapter No	Name of Chapter	Min Marks
1	Fundamentals of Object Oriented Programming	14
2	Foundations of Java	16
3	Inheritance and Package	16
4	Exception Handling and Multithreading	14
5	IO Basics and JDBC	10
Total		70



Fundamentals of Object Oriented Programming

Basics



- **What is mean by Programming language?**
 - ▣ Programming languages can be used to create programs for controlling the behavior of a machine or to express algorithms.
 - ▣ Used to give instructions to computer or computing devices.

Object Oriented Programming



- ❑ Object-Oriented Programming (OOP) is a programming paradigm:
 - ▣ Based on the concept of "objects,"
 - ▣ It can contain data (fields) and methods (functions).

- ❑ Core Concepts: Classes, Objects, Encapsulation, Inheritance, Polymorphism, Abstraction



OOP Vs POP

Feature	POP	OOP
Approach	Focuses on functions and procedures.	Focuses on objects and their interaction.
Data Handling	Data is global and functions operate on it.	Data is encapsulated within objects.
Code Reusability	Limited through functions.	High through inheritance and polymorphism.
Scalability	Less suitable for complex systems.	Well-suited for large, scalable applications.
Security	Limited; data can be freely accessed.	Enhanced with encapsulation and access modifiers.



OOP Vs POP

Feature	POP	OOP
Modularity	Achieved via functions.	Achieved via classes and objects.
Real-World Modeling	Not intuitive for real-world problems.	Natural mapping to real-world entities.
Examples	C, Pascal	Java, C++, Python



Benefits of OOP

- ❑ **Code Reusability:** Allows the use of existing code through inheritance.
- ❑ **Improved Software Maintainability:** Clear structure and modularity make code easier to debug and update.



Benefits of OOP

- ❑ **Encapsulation:** Protects data from unauthorized access and misuse.
- ❑ **Real-World Mapping:** OOP concepts like objects and classes make it easier to model real-world entities.



Benefits of OOP

- ❑ **Flexibility:** Polymorphism allows methods to work with different object types.
- ❑ **Extensibility:** New features can be added with minimal changes to existing code.



Applications of OOP

- ❑ **Graphical User Interface (GUI) Development:** Frameworks like JavaFX and Swing use OOP principles.
- ❑ **Web Development:** Backend programming with servlets, JSP, or frameworks like Spring and Hibernate.



Applications of OOP

- ❑ **Game Development:** Objects can represent players, enemies, and game logic.
- ❑ **Mobile Application Development:** Android apps are primarily developed using Java and OOP concepts.



Applications of OOP

- ❑ **Real-Time Systems:** Simulation and control systems benefit from modular and reusable code.
- ❑ **Database Management:** Integration with OOP principles to model data as objects (e.g., Hibernate ORM).

Basics



C Compilers



☐ GCC

☐ Clang

☐ Microsoft Visual C++

☐ Turbo C

C++ Compilers



☐ G++

☐ Turbo C++

☐ Microsoft Visual C++

☐ Intel C++

☐ Borland C++

☐ Cygwin

C real life applications

Operating Systems (Unix, Linux, and Windows)

Several embedded systems

Video games

Banking systems

Database systems

Basics



C++ real life applications

Adobe systems:

Google: Some Google applications like GFS

Mozilla: developed in C++

MySQL:

Win-amp media player

Symbian OS

Microsoft: Windows 95, 98, 2000, XP,
MS office, IE,

Apple OS

Fundamentals of OOP



- ❑ Object oriented programming structure is programming methodology to organize complex program into simple program by using concepts:

Abstraction

Encapsulation

Polymorphism

Inheritance

Fundamentals of OOP



- ❑ Object oriented programming language which supports abstraction, encapsulation, polymorphism and Inheritance.
- ❑ Ex: C++, Java, C#, VB.NET, Simula-67, Smalltalk, Ruby, Python, etc.

Four Pillars of OOP



Abstraction

Encapsulation

Polymorphism

Inheritance



Four Pillars of OOP

- Abstraction
 - ▣ Hiding unnecessary details and showing necessary details.
 - ▣ Example:



Four Pillars of OOP

- ❑ Encapsulation
 - ▣ Binding data and code together.
 - ▣ Example:
 - ▣ Abstraction changes from one user to another user while encapsulation is same for all users.

Four Pillars of OOP



□ Inheritance

- Creating new class from existing class by acquiring properties (data members) and behaviors(methods/functions) of existing class.
- It is journey from generalization to specialization.
- It provides feature: Reusability. Means we can add additional feature to existing class without modifying it.

Four Pillars of OOP



□ Polymorphism

- ▣ Poly: Multiple, Morphism: forms/behavior
- ▣ Ability to more than one form.
- ▣ Ex: Addition of integers, float, double etc. with same name.
- ▣ Two types: Compile time (Early/static binding) and Run time(Late/dynamic binding).

Class



- ❑ Class is template or blueprint of an object.
- ❑ It is logical entity.
- ❑ Binding data and code together. In case of Java all methods must be composed in class.

Object



- ❑ Object is basic runtime entity.
- ❑ Physical entity.
- ❑ It can be represent: Person, table, place, bank account etc.
- ❑ Object may represent user defined data types: Vectors, Stack, Linked List.
- ❑ Object can be communicate with other object: Customer and Account object can communicate.

Object



□ Characteristics of Object:

▣ State:

- Whatever values data member can contain.

▣ Behavior

- Whatever methods an object can have

▣ Identity

- Object is physical entity, it gets some space in memory or we can identify that object into memory using unique address.



About Java

- ❑ Programming language.
- ❑ Developed by James Gosling and team in 1991 at Sun Microsystems.
- ❑ First version of Java release in 1995.
- ❑ Now, Java is acquired by _____.

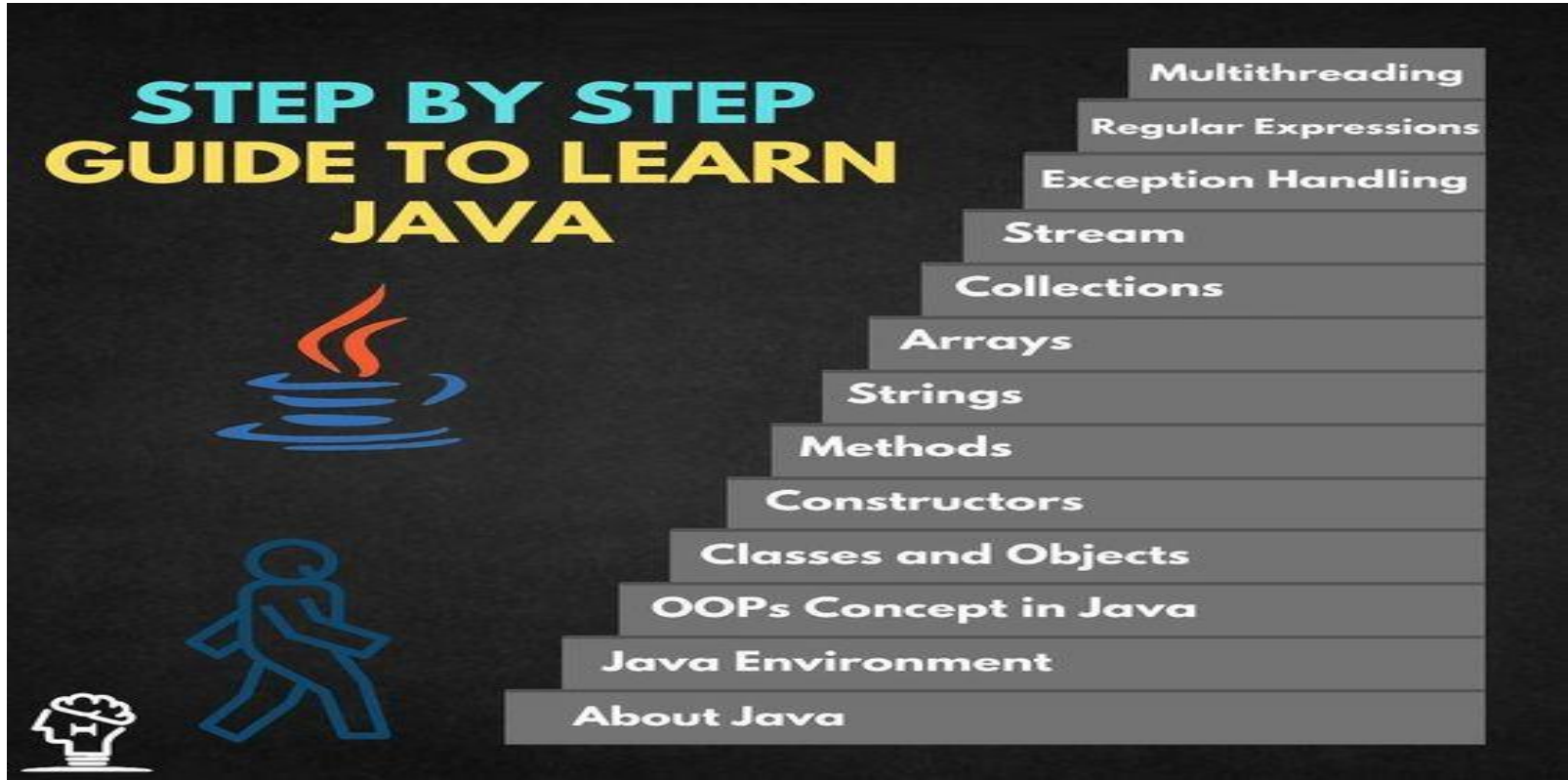


About Java



- ❑ Original name of Java is _____.
- ❑ Multipurpose, object oriented, and cross platform programming language.
- ❑ Powerful programming language.

Roadmap for Java





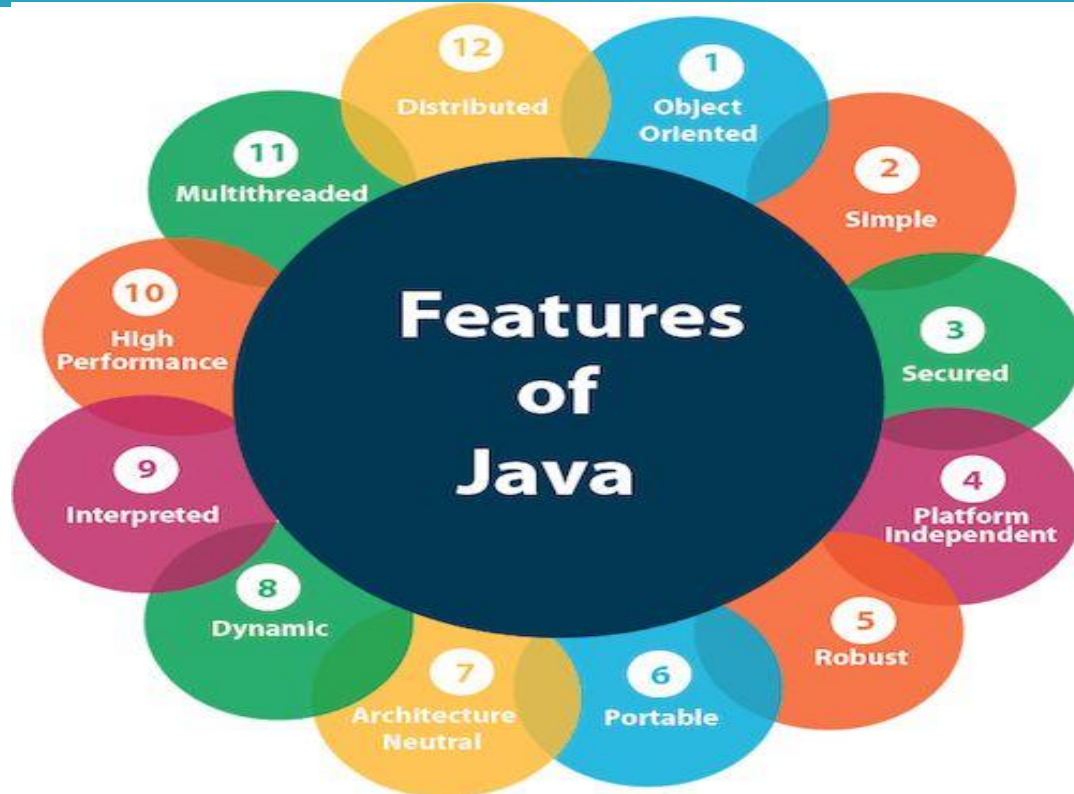
Java

Java Features

Java Features

Important topic in theory exam point of view

Java Features





Java Features

- ❑ Compiled and Interpreted.
- ❑ Platform independent and portable
- ❑ Object oriented
- ❑ Robust and Secure
- ❑ Distributed
- ❑ Multithreading
- ❑ High Performance and so on

Compiled and Interpreted

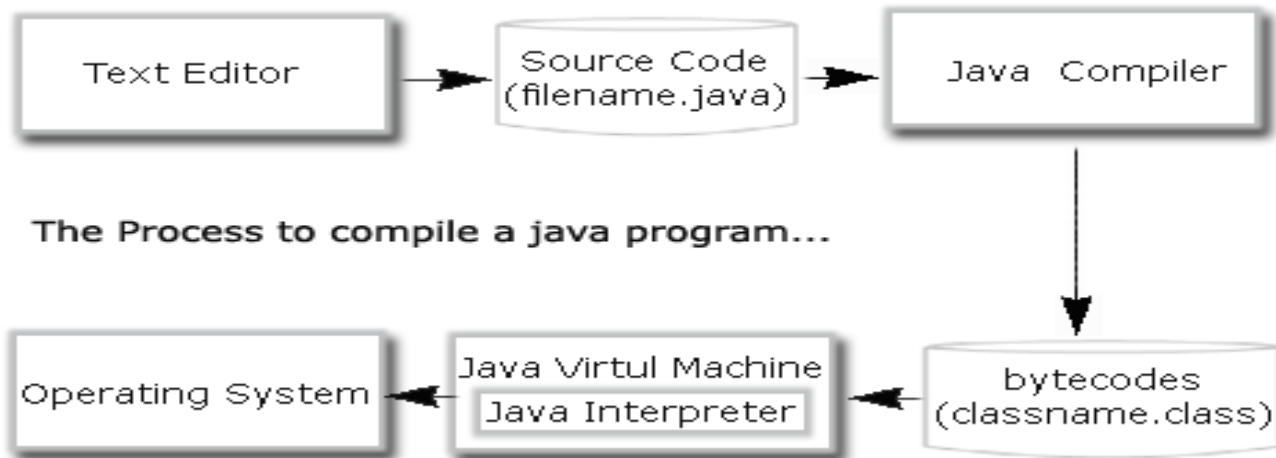


- ❑ Java is both **Compiled and Interpreted language**.
- ❑ First step Java Compiler translates Java code into byte code.
- ❑ Second stage Java interpreter generate machine code from byte code that can be directly executed by machine.

Compiled and Interpreted



- Byte code: Intermediate representation of Java source code. Machine independent.



Static Vs. Dynamic Compilation

▣ Static Compilation:

- “ahead-of-time” (AOT) compilation
- Source code → Native executable.
- Most of compilations work happen before executing.

Static Vs. Dynamic Compilation

- ▣ Modern Java VM dynamic Compilation:

- “Just-in-time” (JIT) compilation
- Source code → Byte Code → Interpreter → JIT executable.
- Most of compilations work happen during application executing.

Platform Independent and portable

- Platform independent because changes and upgrade in operating system, processors and hardware resources will not force to change java program.
- Output of Java compiler is Non executable code i.e. Byte code.
- Portable because byte code which can interpreted to any system with any operating system. Also size of primitive data types are machine independent.

Object Oriented



- ▣ Java is true Object Oriented programming language.
- ▣ Almost everything in Java is in Object.
- ▣ It supports : A-E-I-P.
- ▣ However, it is not 100% OOP language due to availability of primitive data types, does not support multiple inheritance, operator overloading.

Robust and Secure



- It is strictly compile time and run time checking for data types and exceptions.
- Absence of pointers in Java ensures that programs can not gain access to memory location without proper authorization.

Distributed



- ▣ Java is designed as distributed language for creating applications on networks.
- ▣ Allows to share object from remote systems.
- ▣ Java enables multiple program from multiple locations to collaborate and work together to solve problem.

Multithreading



- Multithreading means handling multiple task simultaneously.
Java supports multithreading programming.

High performance



- ▣ Java performance is impressive an interpreted language, mainly due to the use of intermediate byte code.

Java Vs. C++

Java	C++
Does not support Operator overloading	support Operator overloading
Two stage compilation language	Single stage compilation language
Does not support Multiple inheritance directly	Support Multiple inheritance
Java does not uses pointers	It uses pointers
Platform independent	Not Platform independent
Support multithreading	Does not Support multithreading

Java Virtual Machine

JVM task:

Load code

Verify code

Execute code

Provide runtime environment

- ▣ JVM is an abstract machine.
- ▣ It has provide runtime environment in which Java code can be executed.
- ▣ JVM is platform dependent.
- ▣ JVM converts byte code to machine code.
- ▣ JVM provides abstraction between Java program and OS & hardware.

Java Environment



- It includes large number of development tools and thousands of inbuilt classes, interfaces and methods.



JDK (Java Development Kit)

JRE (Java Runtime Environment)

JDK (Java Development Kit)

JDK

Javac (Compiler)

Java (Interpreter)

Javap (Disassembler)

Javadoc (For creating html document)

Javah (For creating C header files)

Jdb (Java debugger)

appletviewer

Sample Java Program

- ❑ Open Notepad and type program.
- ❑ Save program: Program name must be its class name with extension .java
- ❑ Open Command prompt compile program
 - `javac program_name.java`
- ❑ Execute program
 - `java program_name`

About main () method

- ▣ `public static void main(String [ ] args)`
 - `public`: access specifies that makes main method accessible from outside class also.
 - `static`: because interpreter uses this method before object of class is created.
 - `void`: return type of main method which shows main method does not return anything.
 - `String [ ] args`: `args` is array of string which contains all the command line argument.

How to display??

WHICH ONE DID YOU WRITE FIRST ?



<h1>Hello, World!</h1>



printf("Hello, World!");



print("Hello, World!")



cout<<"Hello, World!";



System.out.println("Hello, World!");



console.log("Hello, World!");



println("Hello, World!")



System.Console.WriteLine("Hello, World!");



echo "Hello, World!";



print("Hello, World!")

Naming Conventions

- ▣ Name of all public methods and instance variable start with leading lower case letter. Ex `sum()`, `count()`.
 - When more than one words are used then second word are marked with uppercase letter. Ex `getAge()`, `getRollNumber()` etc
- ▣ All classes and interfaces starts with uppercase letter.
 - Example: `Student`, `Person`
- ▣ Variable that represents constant value use all uppercase
 - Ex `PI`, `TOTAL`

Command Line Argument (CH-02)

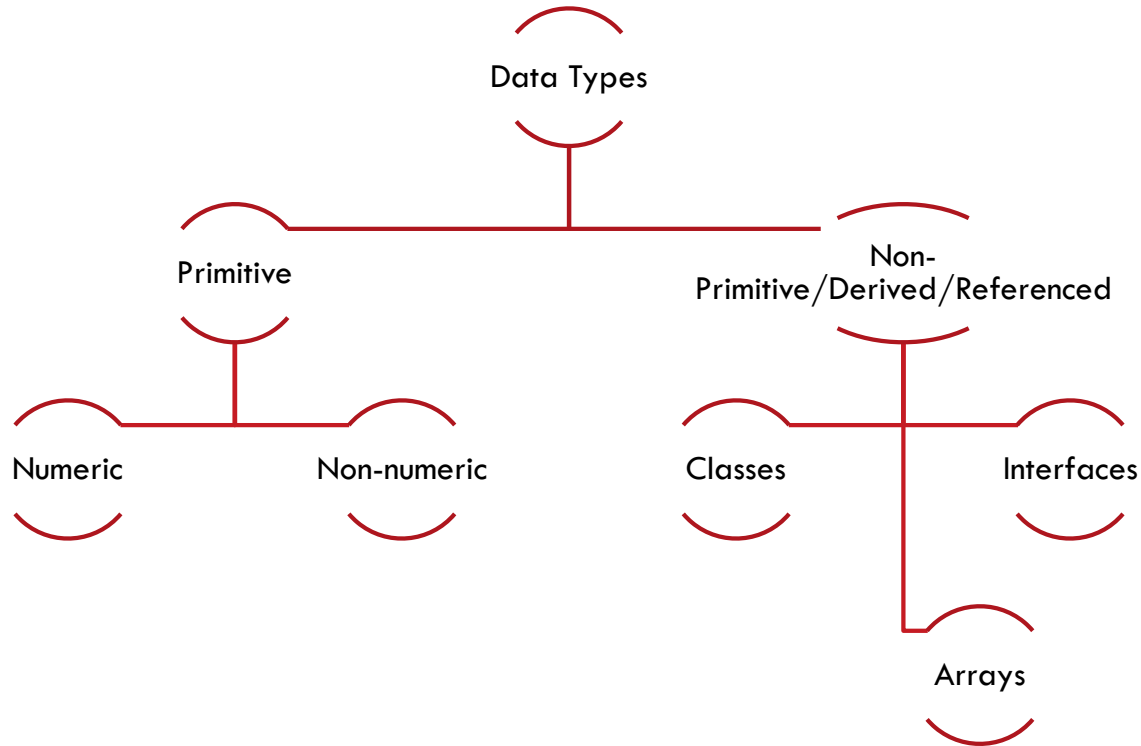
- CLA are parameters that supplied to the application program at the time of execution through command line.
- Example.

Java Tokens and Data types

■ Token:

- Smallest individual units in a program are known as Token.
- Ex: Reserve Keyword, Identifiers, Operators, White space, comments, separators etc.

Java Tokens and Data types



Java Tokens and Data types

■ Data types:

- Type of data hold by variable.
- Two types:
 - Primitive Data type: Numeric and Non Numeric.
 - Referenced Data type/ Non Primitive Data type/Derived
- Primitive Data type:
 - That can hold single value at a time.
 - Its state represent by number, character or true/false. Ex. ??
- Reference/Derived Types:
 - Classes, interfaces and arrays called as reference types in Java.
 - Reference type variable will refer or point to object instead of holding single value.

Data type Detailed Categories

▣ Primitive

■ Numeric

- Integer : byte, short, int, long

- Floating point: float, double

■ Non Numeric

- Character

- Boolean

▣ Non Primitive

■ Classes

■ Interfaces

■ Arrays

Data type Detailed Categories

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Constants

- ▣ Constant variable which do not change its value throughout program execution. Fixed valued once initialize.
 - Numeric Constants
 - Integer Constant : Decimal, Octal and Hexadecimal
 - Real Constants : Ex 12.2, 00.23, 2.45e2
 - Character Constants
 - Character constants
 - String Constants

Variable



- It is an identifier that denotes a storage location used to store a data value.
- Variable may take different values at different time during program execution.
- Naming convention for variable?

Scope of variables

- ▣ Area of program where variable accessible called as scope of variable.
- ▣ Types of variables
 - Instance variables: Declare inside class.
 - Class variables: Declare inside class. Global to class.
 - Local variables : Declare inside method.

Symbolic Constants

- ▣ Symbolic names are written in CAPITALS to distinguish from normal variable
- ▣ Syntax:
 - `final datatype symbolicname = value`
- ▣ Example:
 - `final int PI = 3.14159`

Operators and Expression

- Operator is symbol that tells compiler to perform certain mathematical or logical manipulation.
- Categories:
 - Arithmetic operators : $+$, $-$, $*$, $/$, $\%$
 - Relational operators : $<$, $<=$, $>=$, $>$, $==$, $!=$
 - Logical operators: $\&\&$, $||$, $!$
 - Assignment Operators: $=$
 - Increment and decrement operators : $++$, $--$
 - Conditional operators: $?:$
 - Bitwise Operators
 - Special operators

Operators and Expression

■ Bitwise Operators:

- & Bitwise AND
- ! Bitwise OR
- ^ Bitwise Exclusive OR
- ~ One's Compliment
- << shift left
- >> shift right

x	y	$x \wedge y$
0	0	0
0	1	1
1	0	1
1	1	0

x	y	$x \& y$
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x \mid y$
0	0	0
0	1	1
1	0	1
1	1	1

$$b = a \gg n \longrightarrow b = a / 2^n$$

$$b = a \gg n \longrightarrow b = a * (2^n)$$

Operators and Expression

▣ Special Operators:

- isinstance
- Dot (.)

Decision making

□ if statement:

- Simple if statement
- if – else statement
- Nested if – else statement
- else – if ladder

□ switch statement

Looping

- ▣ While loop
- ▣ Do while loop
- ▣ For loop
 - For each

Jumps in Loops

- ▣ Jumping out of loop : break
- ▣ Skipping part of loop : continue

Mathematical functions

- Available in package

- `min()`

- `max()`

- `sqrt()`

- `pow()`

- `exp()`

- `abs()`

- `round()`